Milestone 3

Search, Sort & Optimize the Catalog

Build-a-Store Semester Project

GOAL	FOCUS	GRADING
Make the catalog searchable, sortable, and more efficient.	Recursion, memoization, binary search, quadratic sorting	70 auto + 30 manual

1. Project Overview

Milestone 2 gave the store a working order-processing system. In Milestone 3, the marketplace grows large enough that finding and organizing products becomes a performance problem. You will add recursive algorithms, a memoized dynamic-programming solution, binary search, and three basic quadratic sorting algorithms.

The goal is to experience the progression from a simple linear solution to algorithms that reduce repeated work or narrow the search space. This milestone builds directly on Milestone 2: keep the existing store, order, stack, queue, and linked-list behavior working while adding the new catalog algorithms.

The progression

Linear search → sort the catalog → binary search. | Repeated recursive work → memoization → dynamic programming.

2. What Stays the Same and What Changes

From Milestone 2	Milestone 3
All M2 classes and behavior remain required.	Add algorithms.py and six new Store methods.
Store keeps its existing products, customers, and orders.	Store gains product sorting, binary-search lookup, recursive catalog total, and change calculation.
LinkedList, Stack, and OrderQueue continue to work as before.	Do not modify their required behavior for this milestone.

3. New Algorithms and Their Purpose

Algorithm	Purpose
Recursive catalog total	Compute the total value of all product prices using recursion.
Fewest coins	Use top-down memoization to find the minimum number of coins needed for a cash amount.
Bubble sort	Sort products by product ID using bubble sort.
Selection sort	Sort products by product ID using selection sort.
Insertion sort	Sort products by product ID using insertion sort.
Binary search	Search a product list already sorted by product ID in O(log n) time.

4. Required Classes, Functions, and Methods

Follow the function and method names, parameters, and return types exactly. You may add helper functions, but do not rename or change the required methods.

4.1 New algorithms.py

Put the six required functions below in algorithms.py. The sorting functions must work with Product objects and must modify the given list in place. Do not use Python's built-in sort() or sorted() for the three required sorting algorithms.

Function	Purpose	Return type
recursive_total_price(products)	Return the total of all product prices using recursion. Base case: empty list returns 0.0. Otherwise, return the first product's price plus recursive_total_price() on the remaining products. Do not use a loop. For example, for [P1, P2, P3], the function returns P1.price + recursive_total_price([P2, P3]). The calls continue until the remaining list is empty, which returns 0.0.	float
fewest_coins(coins, amount)	Return the minimum number of coins needed to make amount using top-down memoization.	int
bubble_sort(products)	Sort the Product list in place in ascending order by product ID. Implement the named quadratic sorting algorithm; do not use sort() or sorted().	None
selection_sort(products)	Sort the Product list in place in ascending order by product ID. Implement the named quadratic sorting algorithm; do not use sort() or sorted().	None
insertion_sort(products)	Sort the Product list in place in ascending order by product ID. Implement the named quadratic sorting algorithm; do not use sort() or sorted().	None
binary_search(products, product_id)	Search a list sorted by product ID; return the matching index or -1.	int

4.2 Store Updates

All Milestone 2 methods remain required. In store.py, import the required functions from algorithms.py. Each method below must call its corresponding function from algorithms.py and use the result; do not reimplement the algorithm inside Store.

Method	Purpose	Return type
bubble_sort_products()	Call bubble_sort(self.products) from algorithms.py. The algorithm sorts the store's products in place by product ID.	None
selection_sort_products()	Call selection_sort(self.products) from algorithms.py. The algorithm sorts the store's products in place by product ID.	None
insertion_sort_products()	Call insertion_sort(self.products) from algorithms.py. The algorithm sorts the store's products in place by product ID.	None

binary_search_product(product_id: str)	Call binary_search(self.products, product_id) from algorithms.py. The products must already be sorted by product ID. If the returned index is -1, return None; otherwise return that Product.	Product | None
recursive_catalog_total()	Call recursive_total_price(self.products) from algorithms.py and return the value it returns.	float
calculate_change(coins: list[int], amount: int)	Call fewest_coins(coins, amount) from algorithms.py and return the value it returns.	int

5. Example Input, Output, and Expected Behavior

In this project, "input" means method calls and objects passed to your functions or classes. "Output" means returned values or observable changes in the objects. You do not need to use input().

Example 1: Sort and Binary Search

```
store = Store()

mouse = Product("P300", "Mouse", 29.99)
keyboard = Product("P100", "Keyboard", 59.99)
headphones = Product("P200", "Headphones", 39.99)

store.add_product(mouse)
store.add_product(keyboard)
store.add_product(headphones)

store.insertion_sort_products()
print([p.get_id() for p in store.products])  # Output: ['P100', 'P200', 'P300']

product = store.binary_search_product("P200")
print(product.get_name())  # Output: Headphones

missing = store.binary_search_product("P999")
print(missing)  # Output: None
```

Example 2: Recursive Total and Fewest Coins

```
print(store.recursive_catalog_total())  # Output: 129.97

coins = [1, 5, 10]
print(store.calculate_change(coins, 12))  # Output: 3
print(store.calculate_change(coins, 0))   # Output: 0
```

Example 3: The Three Sorting Algorithms

```
products = [mouse, keyboard, headphones]

for sorter in [bubble_sort, selection_sort, insertion_sort]:
    sorter(products)
    print([p.get_id() for p in products])  # Output: ['P100', 'P200', 'P300']
```

6. Required Testing with unittest

Submit test_milestone3.py using Python's built-in unittest framework. Test observable behavior, not private implementation details.

- Import unittest and the functions/classes you are testing.
- Create test classes that inherit from unittest.TestCase.
- Every test method must start with test_.
- Use assertions to check returned values and observable behavior.
- Do not inspect private variables inside algorithms or Store implementation.
- Test the algorithms directly and test Store methods as integration behavior.
- Include normal and edge cases: empty lists, one product, product found/not found, amount 0, impossible change, already-sorted input, and reverse-sorted input.

Test area	Minimum tests	Examples
Recursion / catalog total	2	empty list, multiple products
Fewest coins	3	amount 0, exact coin, larger memoized amount
Sorting	4	bubble, selection, insertion, already sorted/reverse order
Binary search	3	first item, middle item, not found
Store integration	3	sort integration, binary lookup, change calculation

Unit-test expectation

Target: at least 15 meaningful tests. The unit-test file is worth 20 points and will be reviewed manually by the TA. Meaningful coverage and correct use of unittest matter more than simply reaching the minimum count.

7. Running-Time Analysis

Submit a plain-text file named complexity.txt. For each operation below, provide its worst-case Big-O running time. Use the exact operation names shown here.

- recursive_total_price
- fewest_coins (memoized)
- bubble_sort
- selection_sort
- insertion_sort
- binary_search
- Store.bubble_sort_products
- Store.selection_sort_products
- Store.insertion_sort_products
- Store.binary_search_product

8. Gradescope Submission

This milestone is cumulative. Submit the working Milestone 2 files plus the new algorithms module, tests, and complexity file.

File	Status in M3	What it contains
product.py	Same as M2	Product class from previous milestones.
customer.py	Same as M2	Customer class from previous milestones.
cart.py	Same as M2	ShoppingCart behavior from previous milestones.
order.py	Same as M2	Order class from Milestone 2.
node.py	Same as M2	Node used by LinkedList.
linked_list.py	Same as M2	LinkedList implementation.

stack.py	Same as M2	Stack ADT using LinkedList.
order_queue.py	Same as M2	Queue ADT using LinkedList.
store.py	Updated	Adds the six new Milestone 3 methods.
algorithms.py	New	Six required algorithms.
test_milestone3.py	New	Your unittest tests; manually graded.
complexity.txt	New	Ten required Big-O answers; manually graded.

Gradescope submissions: You may submit as many times as you would like before the deadline. Gradescope will automatically grade program functionality and provide feedback after each submission. The unittest file and complexity.txt are reviewed manually by the TA. Your final submission before the deadline is used for grading.

9. Code Documentation

Include a short, meaningful docstring for each class and required function. Use brief comments when needed to explain important or non-obvious code. Keep comments focused; do not simply restate every line of code.

10. Post-Milestone Feedback & Peer Evaluation

After each milestone, every student must complete an individual contribution and peer-evaluation form describing their own contributions, their teammate's contributions, teamwork and communication, and any resolved or unresolved issues. Although the project is completed as a team, grades may differ based on individual contribution.

11. Grading

Component	Description	Points
Recursive / memoized algorithms	Produces correct results for recursive total-price and fewest-coins operations. Automatically graded.	15
Sorting algorithms	Produces correct sorted results using the required sorting operations. Automatically graded.	15
Binary search	Correctly finds products and handles not-found cases. Automatically graded.	10
Store integration	Correctly connects sorting, binary search, recursive total, and change calculation to Store. Automatically graded.	20
Algorithm implementation & design	Uses the required algorithms: recursion, memoization, binary search, bubble sort, selection sort, and insertion sort. Store methods correctly call the corresponding functions from algorithms.py. **Manually graded.**	25
Unit tests	Provides at least 15 meaningful unittest tests covering the required areas and testing observable behavior. **Manually graded.**	10
Running-time analysis	Provides the required worst-case Big-O answers in complexity.txt. **Manually graded.**	5
Total		100

Final Submission Checklist

- All Milestone 2 functionality still works.
- algorithms.py contains all six required functions.
- Store contains all six required Milestone 3 methods.
- The three required sorting algorithms do not use sort() or sorted().
- binary_search() is used on product lists sorted by product ID.
- test_milestone3.py contains at least 15 meaningful tests.
- complexity.txt contains all 10 required Big-O answers.
- Every class and required function has a short docstring.

Looking Ahead

The store can now search a sorted catalog in O(log n), but it must first maintain that ordering. In the next milestone, the catalog will grow much larger, and we will explore divide-and-conquer sorting and hashing so the store can scale to much larger collections.

Milestone 3 goal

Make the catalog easier to search and organize, and understand how recursion, memoization, and algorithm choice affect performance.